Is there any control or tuning in BLEAK for python to help establish connections. The connection establishment =pairing is not as consistent and reliable from the PC running BLEAK compared to a a link between my silicon labs efr32 by cusint device and and a SiLabs WSTK acting as a BLE client

Yes—but the controls exposed by Bleak are fairly limited. Bleak is primarily a cross-platform wrapper over the host OS Bluetooth stack; on a Windows PC, the WinRT Bluetooth stack decides most link-layer behavior: scan scheduling, connection initiation timing, connection interval, PHY, supervision timeout, and much of pairing/bonding. So it is very plausible for an EFR32 ↔ WSTK link to be much more repeatable than EFR32 ↔ Windows/Bleak, even with the same peripheral firmware.

What you can tune in Bleak

The most useful improvements are:

1. **Scan first and pass the resulting** `BLEDevice` **to** `BleakClient`—do not normally construct the client from a raw MAC address.
2. Use `pair=True` if the target needs a bond/security before service discovery or protected-characteristic access.
3. Make the timeout long enough, especially during first pairing.
4. Keep all BLE work in one continuously running `asyncio` event loop.
5. Add retry/backoff and ensure every failed connection path cleans up.
6. If possible, restrict service discovery to the service UUID(s) you need.

Bleak documents that a string address causes an implicit discovery operation; using a scanned `BLEDevice` avoids that extra discovery path and is its recommended model for more complex cases. It also supports `pair=True`, an explicit `timeout`, an optional service UUID filter, and a disconnect callback.[1]

```
import asyncio
from bleak import BleakClient, BleakScanner
from bleak.exc import BleakError

TARGET_NAME = "My_EFR32_Device"
MY_SERVICE_UUID = "12345678-1234-5678-1234-56789abcdef0"
```

```
async def find_target():
    # Use a longer, explicit scan so the PC has recently seen a connectable ADV.
    return await BleakScanner.find_device_by_name(
        TARGET_NAME,
        timeout=12.0,
    )

async def connect_once(device):
    def disconnected(client):
        print(f"Disconnected from {client.address}")

    async with BleakClient(
        device,                         # BLEDevice, not a bare MAC string
        pair=True,                      # If a bond/authentication is required
        timeout=60.0,                   # Important for first-time pairing/UI
        services=[MY_SERVICE_UUID],     # Optional; may speed enumeration on Windows
        disconnected_callback=disconnected,
    ) as client:
        print("Connected:", client.is_connected)
        # Do protected reads/writes after pairing/connect has completed.
        # await client.read_gatt_char(...)
        await asyncio.sleep(1)

async def main():
    for attempt in range(1, 6):
        device = await find_target()

        if device is None:
            print(f"Attempt {attempt}: device not found")
            await asyncio.sleep(min(2 ** attempt, 15))
            continue

        try:
            await connect_once(device)
            return
        except (BleakError, asyncio.TimeoutError, OSError) as e:
            print(f"Attempt {attempt} failed: {e!r}")
            await asyncio.sleep(min(2 ** attempt, 15))

    raise RuntimeError("Could not establish a BLE connection after retries")

asyncio.run(main())
```

`pair=True` asks Bleak to pair before connecting if the device is not already paired. It does **not** give Python direct control of the Security Manager pairing sequence; Windows owns the UI, method selection, key storage, and bond state. If pairing requires a PIN/passkey or user confirmation, increase `timeout` substantially beyond the normal connect time. [1]

Connection is not pairing

It helps to separate three things that often get conflated:

Stage	What it means	Who mostly controls it on PC
Advertising / scanning	The EFR32 advertises; Windows happens to hear it and identify it	EFR32 advertising parameters + Windows scanner
Link connection	Central sends the connection request; a BLE ACL link comes up	Windows controller/stack, not Bleak
Pairing / bonding	Security procedure establishes keys; bond persists those keys	Windows pairing stack + EFR32 security configuration
GATT discovery/access	Windows enumerates services and reads/writes characteristics	Bleak calls plus Windows GATT implementation

So a problem that looks like “pairing inconsistency” may actually be any of the following:

- The Windows radio does not see—or does not attempt to connect promptly to—the advertising packet.
- The peripheral is no longer in a connectable advertising state.
- Service discovery begins before a required security state has been reached.
- Windows and the EFR32 disagree about a stored bond/key.
- The EFR32 has exhausted or replaced bond slots.
- Another application, phone, WSTK, or stale PC connection is still holding or racing for the peripheral.
- The PC-side driver/radio has a Windows-specific timing or coexistence problem.

A Bleak/Windows discussion found cases where long advertising intervals—over roughly one second—produced very long or failed Windows connections, and no Bleak connection parameter was available to make Windows behave like another central. [2]

EFR32-side changes likely to help

Because the WSTK succeeds, I would first compare the **air behavior and security state** rather than assume your application logic is wrong. For your EFR32 peripheral, consider these practical changes.

Use a fast connectable advertising phase

At boot, after a disconnect, and whenever the peripheral is awaiting a new central, advertise connectably at a relatively short interval—for example, commonly on the order of 20–100 ms—before dropping to a slow low-power interval after a timeout.

This is especially important for Windows. If your device switches quickly to 1–3 s advertising, the PC may appear flaky even though a WSTK central works consistently. The Bleak Windows report specifically correlated greater-than-one-second advertising intervals with extreme connection delays. [2]

Also check:

- ADV type is truly **connectable** in the state where the PC tries to connect.

- You are not alternating into a nonconnectable beacon/telemetry mode.
- The local name and/or service UUID is present consistently enough for your scan filter.
- The device does not stop advertising or enter sleep while Windows is still performing its scan-to-connect path.
- It resumes advertising promptly after a failed security procedure or disconnect.

Make bonding deterministic

For a development system, stale bonds are an extremely common source of "works with WSTK but not with Windows" symptoms.

Use a deliberate test matrix:

1. Erase bonds on the EFR32.
2. Remove/unpair the device in Windows Bluetooth settings.
3. Restart the EFR32 advertising state.
4. Pair once from the intended Windows/Bleak workflow.
5. Confirm reconnect behavior without re-pairing.
6. Repeat while logging the EFR32 security events and the Windows/Bleak exception.

On a production device, define what happens when bond storage is full: reject new bonds cleanly, evict an old one by a documented policy, or provide a physical/software "clear bonds" procedure. If Windows presents an old LTK while the EFR32 has forgotten that bond, the resulting encryption failure can look like random connection failure.

If your GATT attributes require encryption or authentication, make sure the EFR32 security policy matches the pairing capability you expect Windows to use:

- **Just Works** is simplest for reliability testing but provides no MITM protection.
- **Passkey entry / numeric comparison** needs a coherent user interaction path on both sides.
- **LE Secure Connections** should be consistently enabled/configured as intended.
- Avoid requiring a protection level that the device cannot actually complete—for example, a display/passkey flow on a headless device with no usable input/output.

Windows-specific checks

Avoid the wrong asynchronous model

On Windows, Bleak's WinRT backend needs an MTA execution environment for a normal console application. Imports involving `pywin32`/`pythoncom` can initialize the process as STA, which can prevent async WinRT callbacks from completing correctly. Bleak recommends setting `sys.coinit_flags = 0` before an import that triggers `pythoncom`, or using its `uninitialize_sta()` helper when an unwanted STA initialization has already occurred. [3]

If your program uses `pywin32`, COM automation, Office automation, certain GUI frameworks, or a notebook/IDE integration, put this at the **very top** of the program before those imports:

```
import sys
sys.coinit_flags = 0  # MTA for normal console-style WinRT/Bleak use

import asyncio
from bleak import BleakClient, BleakScanner
```

If something has already initialized STA:

```
try:
    from bleak.backends.winrt.util import uninitialize_sta
    uninitialize_sta()
except ImportError:
    pass
```

Do not use `uninitialize_sta()` if you intentionally run a real GUI application whose UI message loop is correctly integrated with asyncio; that is a different case.[3]

Use one `asyncio.run()`

Do not scan in one `asyncio.run()` call and connect in another, nor invoke `asyncio.run()` inside a retry loop. Bleak explicitly requires a single running asyncio loop for all its operations; repeated `asyncio.run()` calls can cause intermittent crashes and failures. Use `await asyncio.sleep(...)`, not `time.sleep(...)`, within async code.[3]

Upgrade and test the physical host radio

- Update Bleak and Python, plus the PC's Bluetooth driver from the PC/OEM or chipset vendor.
- Test with an external USB Bluetooth adapter placed away from the PC chassis and USB 3 noise sources.
- Temporarily disable Wi-Fi as an interference/coexistence experiment, especially if the Bluetooth radio is integrated with Wi-Fi.
- Make sure the WSTK, a phone, Windows Settings, Chrome, Bluetooth LE Explorer, or another test program is not simultaneously trying to connect.
- If the peripheral only permits one central connection, verify the prior central disconnected fully before testing the PC.

Bleak notes that simultaneous scanning and active connections—and the number of connections possible—depend on the adapter hardware.[1]

Diagnose with traces

Enable Bleak logging first:

```
$env:BLEAK_LOGGING=1
python your_program.py
```

Then capture a good WSTK connection and a bad Windows connection. The key is to identify **where the sequence diverges:**

Observed result	Most likely focus
PC sees ADV but no `CONNECT_IND` for a long time	Advertising interval, Windows scan/connect behavior, PC adapter/driver
`CONNECT_IND` occurs but link immediately terminates	Connection parameters, RF/coexistence, peripheral resource/state handling
Link connects but pairing fails	IO capability, security requirements, stale/invalid bond keys, bond database
Pairing completes but service discovery fails	Security permissions, GATT DB/cache, service configuration
First connect works, later reconnect fails	Bond persistence, peripheral bond-slot policy, disconnect/advertising restart

Bleak's troubleshooting guide includes Windows Bluetooth traffic capture through the Windows Bluetooth Test Platform and Wireshark, which is the most direct way to distinguish a host-side initiation delay from an SMP/GATT failure. [3]

Bottom line

The closest things to "tuning" in Bleak are:

```
device = await BleakScanner.find_device_by_name(NAME, timeout=10)
async with BleakClient(
    device,
    pair=True,
    timeout=60,
    services=[YOUR_SERVICE_UUID],
) as client:
    ...
```

But there is **no Bleak knob for Windows connection interval, scan duty cycle, connection-request timing, PHY, or supervision timeout**. For a robust EFR32 ↔ Windows design, the highest-value work is usually: fast connectable advertising during the connection window, clean one-time bond management, a scan-then-connect architecture using `BLEDevice`, one asyncio loop/MTA on Windows, and HCI/air capture comparison against the known-good WSTK case.

⁂

1. https://bleak.readthedocs.io/en/latest/api/client.html

2. https://github.com/hbldh/bleak/discussions/910
3. https://bleak.readthedocs.io/en/latest/troubleshooting.html
4. https://superuser.com/questions/1423299/bug-in-windows-bluetooth-le-getgattservicesasync-method
5. https://github.com/microsoft/Windows-universal-samples/issues/764
6. https://github.com/hbldh/bleak/discussions/933
7. https://bleak.readthedocs.io/_/downloads/en/develop/pdf/
8. https://bleak.readthedocs.io/en/latest/api/index.html
9. https://www.reddit.com/r/esp32/comments/1i7trwg/esp32_and_bluetooth_low_energy_ble/
10. https://learn.microsoft.com/en-us/answers/questions/3192714/problem-with-bluetooth-low-energy-gatt-compliant-h
11. https://stackoverflow.com/questions/74426593/how-to-adjust-the-time-to-connect-with-python-bleak-module
12. https://stackoverflow.com/questions/67575000/in-bluetooth-le-gatt-is-there-any-way-to-detect-when-long-term-keys-are-invalid
13. https://learn.microsoft.com/en-us/windows/apps/develop/devices-sensors/bluetooth-dev-faq
14. https://blogs.windows.com/windowsdeveloper/2017/01/13/new-bluetooth-features-in-creators-update-gatt-server-bluetooth-le/
15. https://github.com/hbldh/bleak/issues/1880